GLM-4-Voice（arxiv, 2024.12, 智谱AI, Github:https://github.com/THUDM/GLM-4-Voice）

目的：

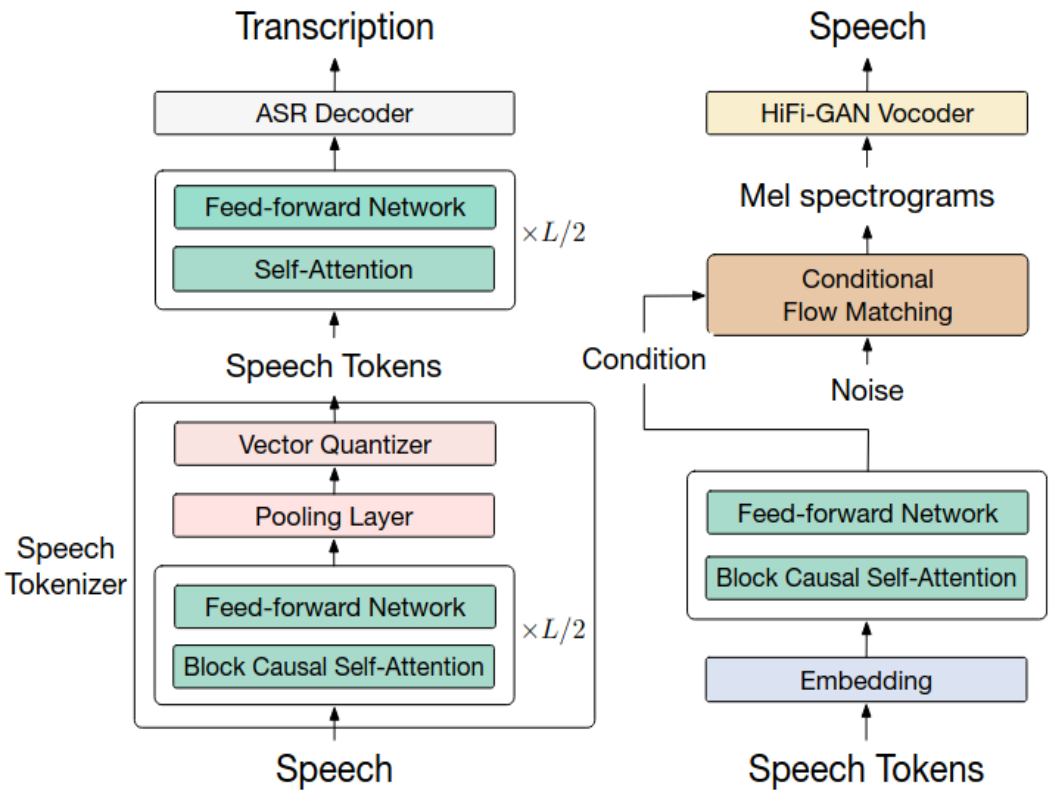
基于语音的交互为人机交互提供了一种更自然、更直观的媒介，从而提供更丰富、更具吸引力的用户体验。我们的目标是构建一个具有高智能的类人**端到端**语音聊天机器人。

- 分词器：GLM-4-Voice 使用超低比特率 （175bps）、具有 12.5Hz 帧速率的单码簿语音分词器。里面还有一个矢量量化的模块进一步缩短语言的向量表示模块。比特率越高音频越清楚。这个地方的功能还没确认。
- \item 分词器：分词器负责将一系列词元映射为最终的文本字符串 (例如 [1169, 3797, 3332] -> “the cat sat”)。GLM-4-Voice 使用超低比特率 （175bps）、具有 12.5Hz 帧速率的单码簿语音分词器。里面还有一个矢量量化的模块进一步缩短语言的向量表示模块。比特率越高音频越清楚。这个地方的功能还没确认。力和语音质量。

之前工作没有克服的困难：

1. 语音数据相较与文本数据是稀缺的，这种数据不平衡使得基于语音数据训练的大模型很难达到像chatgpt模型类似的智能程度，最终限制了 SpeechLM 的智能。
2. 其他方法通过将speech encoder和text-to-speech模块集成到现有的LLM中，并在口语对话数据集上对其进行微调，来使语音和文本模态保持一致（omni文章的做法）。相当于克服了语料数据不足的问题。由于缺乏专门的语音预训练，它缺乏提供真正类似人类的语音输出的能力。没有办法像人类一样输出带有语气、语调的语音。（意思是说，之前的文章只是用简单的语音对话模型对LLM进行了微调，训练的不够。本文用了各种数据对模型进行预训练，比方说Automatic Speech Recognition (ASR), LLM processing, and Text-to-Speech (TTS)方面的数据库）。

Method：



1. Speech Tokenizaion：连续的语音信息$\rightarrow$离散的token，包含语义信息和部分声学信息。

这里主要是让token的信息足够丰富。基于重建的方法存在一定的问题。（这种方法导致声学细节的丢失，且依赖高采样率）。

结构：whisper-large-v3的Encoder+池化层+向量量化层。

Encoder中Transformer之前的卷积被替换为因果卷积（causal convolution），Encoder中的bidirectional attention被换为block causal attention，结构的变化帮助去引入因果关系。这些调整帮助实现在推理期间，启用输入语音的流式编码。

Encoder是在预训练阶段通过ASR任务获得，而后我们在Encoder后面加上池化层和向量量化层去微调获得Tokenizer。在微调的过程中建立codebook（token_id$\xleftrightarrow{词汇表}$token$\leftrightarrow$（一段语音or文字字符））。

> 按理来说加了向量量化层，那么Tokenizer输出的是token_id，如果输入是index那么LLM的输出是什么也是token_id吗？？？？

2. Speech Decoder：从离散的语音标记合成语音波形。Decoder architecture of CosyVoice作为我们文章的解码器。包括speech token encoder, a conditional flow matching model, and a HiFi-GAN vocoder（声码器）。在这个模块，结合生成的token和codebook生成语音。

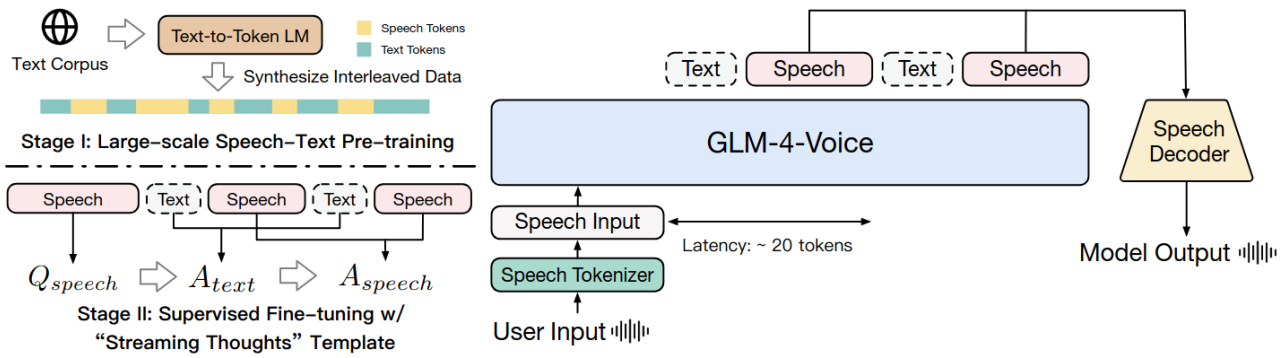
   训练细节：从0开始训练speech token encoder，flow matching model采用两阶段训练范式，以充分利用质量参差不齐的丰富语音数据。预训练阶段使用各种无监督语音数据。在微调阶段，使用单个speaker的高质量语音数据。

Inference

考虑到大语言模型的成功和文本包含大部分的语音信息，所以我们把Speech-to-Speech task分解为speech-to-text and speech-and-text-to-speech。定义Speech input：Q_s，对应的文本回复A_t，Speech输出A_s。

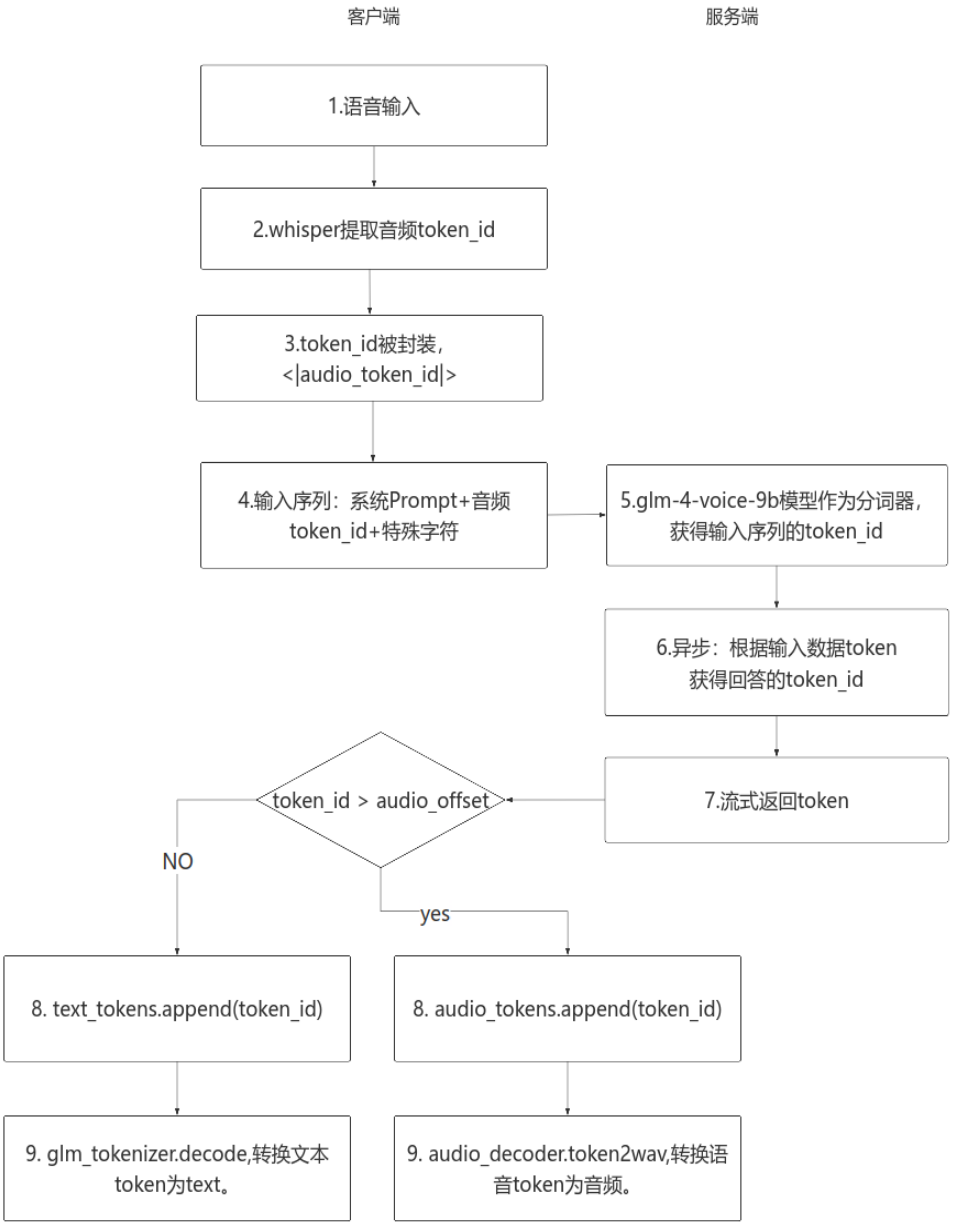
1. Speech-to-Text：$Q_s \rightarrow A_t$
2. Speech-and-Text-to-Speech：$Q_s + A_t \rightarrow A_s$，$A_s$具有自适应语气和韵律，以确保对话的连贯性。

这么做存在一个问题，A_s的输出要等到A_t完全输出，才能进行。因此文章，提出a_t和a_s交替输出，最后在组合各个阶段的a_t和a_s为A_s，A_t。



关于训练过程的思考

代码的推理细节




```
# <|audio_10815|><|audio_5966|><|audio_4417|>....
# 这么看来audio被转换为index
```

第三步，整体内容的token_id

```
inputs = {'input_ids': tensor([[151331, 151333, 151335,
  > special variables
  > function variables
  data = {'input_ids': tensor([[151331, 151333, 151335,  ...
    > special variables
    > function variables
    > 'input_ids' = tensor([[151331, 151333, 151335,    198,...
    > 'attention_mask' = tensor([[1, 1, 1, 1, 1, 1, 1, 1, 1,...
    > 'position_ids' = tensor([[  0,   1,   2,   3,   4,   5...
```

LLM的输出：包括文本信息和音频信息，统一用token标识

```
> chunk = b'{"token_id": 1513...
  186    text_tokens.appen
  187
v complete_tokens = [98384, ...
  > special variables
  > function variables
    [98384, 99186, 252, 3837, 101352, 98860, 105976, 100629, 119382, 3837 ...ch
    Copy Value   Copy as Expression
WATCH
  190    complete_text = glm_tokenizer.dec
```

LLM输出做一次decoder

```
  complete_text = '好嘞，重...
  181    if not is_finalize:
> complete_tokens = [983
  end_token_id = 151336
    '好嘞，重庆红崖洞这个地方，那可是相当<|audio_10815|><|audio_1321|><|audio_6...
    Copy Value   Copy as Expression
  flow_prompt_speech_token = t
```